

- Abstract classes
- Interfaces
- Annotations



## # Abstract class

Sometimes you will want to create a superclass that only defines a generalized form that will be shared by all of its subclasses, leaving it to each subclass to fill in the details. Such a class determines the nature of the methods that the subclasses must implement.

You may have methods that must be overridden by the subclass in order for the subclass to have any meaning. In this case, you want some way to ensure that a subclass does, indeed, override all necessary methods. Java's solution to this problem is the abstract method.

You can require that certain methods be overridden by subclasses by specifying the abstract type modifier.

```
abstract type name(parameter-list);
```

These methods are sometimes referred to as subclass's responsibility because they have no implementation specified in the superclass.

Thus, a subclass must override them—it cannot simply use the version defined in the superclass.

★ → Any class that contains one or more abstract methods must also be declared abstract.

# There can be no objects of an abstract class.

# You cannot declare abstract constructors, or abstract static methods.

static methods cannot be overridden then there's no point to create static method abstract.

# You can declare static methods in abstract class.

Because there can be no objects for abstract class. If they had allowed to call abstract static methods, it would mean we are calling an empty method (abstract) through classname because it is static.

Any subclass of an abstract class must either implement all of the abstract methods in the superclass, or be declared abstract itself.

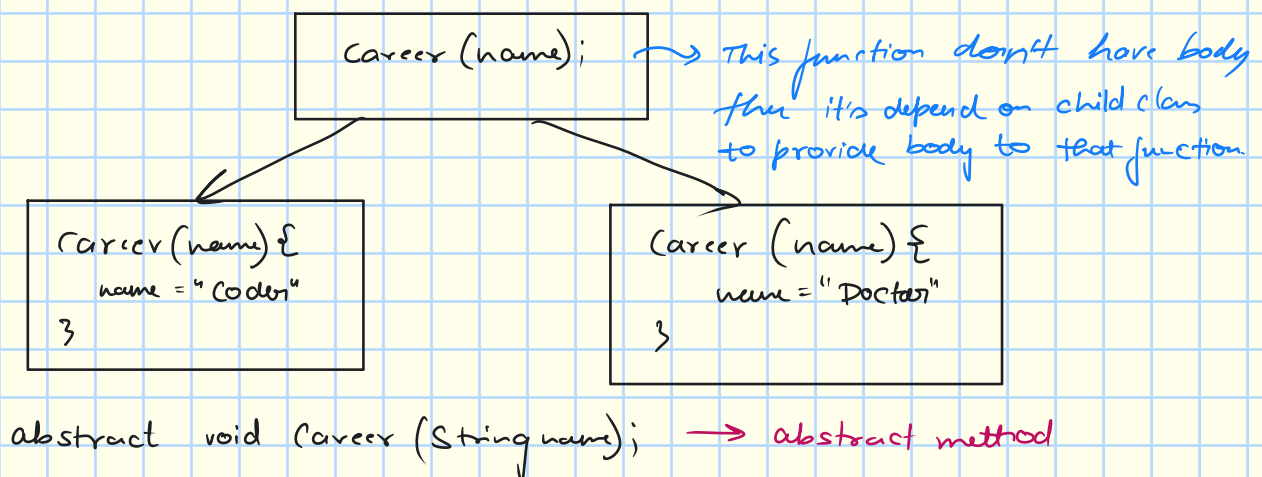
Abstract classes can include as much implementation as they see fit i.e. there can be concrete methods (methods with body) in abstract class.

Although abstract classes cannot be used to instantiate objects, they can be used to create object references, because Java's approach to run-time polymorphism is implemented through the use of superclass references.

A public constructor on an abstract class doesn't make any sense because you can't instantiate an abstract class directly (can only instantiate through a derived type that itself is not marked as abstract)

Check: <https://stackoverflow.com/questions/260666/can-an-abstract-class-have-a-constructor>

→ If class contains one or more abstract method, then class also be declared as abstract.



Abstract class: → • In Java, an abstract class that cannot be instantiated & is designed to be extended by other classes. can't make

- It is used to achieve partial **abstraction**, where some methods are implemented while others are left for subclasses to define.
- An abstract class is declared using the **abstract** keyword.

It may contain:

- **Abstract method** (methods without body)
- **Concrete method** (methods with implementation)
- **Constructors**
- **Instance variables**
- **static & final methods.**

ex:- abstract class class-name {  
    abstract void method\_name(); // Abstract method  
    void concrete\_method() {  
        Sout("Method implementation");  
    }  
}

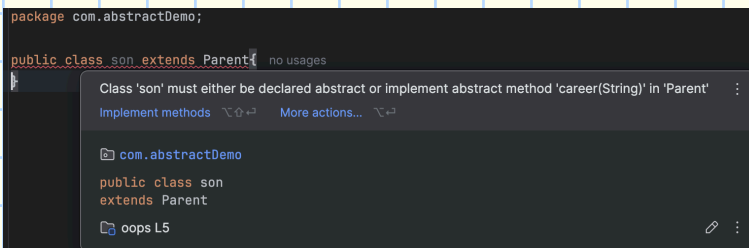
} } Concrete method

ex:-

```
abstract class Shape {  
    int color;  
    int getColor() { return color; }  
    Shape (int color) {  
        this.color = color;  
    }  
    abstract double getArea();  
}  
  
class Square extends Shape {  
    double side;  
    Square (int color, double side) {  
        super (color);  
        this.side = side;  
    }  
    double getArea() { return side * side; }  
}  
  
class Test {  
    psvm(...){  
        Square s = new Square(5, 10.0);  
        Sout(s.getArea()); // 100.0  
    }  
}
```

- \* No Instances
- \* Reference can be there of abstract class
- \* Can have constructor
- \* A class can be abstract without any abstract method
- \* we can have static methods
- \* If a class has at least one abstract method, the class must be made abstract
- \* If a subclass does not implement all abstract method, then the derived class must also be abstract.

NOTE:- Every child class have to override all the methods present in parent class. as those method not have body. if child class not override all abstract method will throw error.



{Parent → abstract class}

→ Why we cannot make object of abstract class?

If you are able to make object, then assume you called abstract method, then it'll give error it doesn't have any body, how can I call it.

NOTE: → We cannot make constructor abstract.

# Abstract Constructor :-> cannot make.

# Object of an Abstract class :-> cannot make object.

# Abstract static method :-> No, as abstract method mean to be overridden & static prevent from overriding.

# Static method in Abstract class :-> Yes possible

ex:-

```
abstract class Parent {  
    static void hello() {  
        |  
        sout("Hey");  
    }  
}
```

```
public class main {  
    psvm {  
        |  
        Parent.hello(); // Hey  
    }  
}
```

# Normal method in Abstract class :-> Yes we can, that method will be override in child class & then we can use through object of child class

# Using abstract class as Reference

Parent-class object-name = new child-class();

# "final" Keyword in Abstract class :-> Not possible becoz final prevent inheritance

but can have final Variable.

ex:-

```
public abstract class Parent {  
    int age;  
    final int VALUE;  
    public Parent(int age) {  
        this.age = age;  
        VALUE = 32456789;  
    }  
}
```

```

abstract class engine {
    int a = 10;
    abstract void start();
}

```

```

car extends engine {
    int b = 99;
    @Override
    public void start() {
        sout("start car");
    }
}

```

reference  
type

```

(car) car = new (car());
sout(car.a); // 10
sout(car.b); // 99
car.start; // start car

```

object  
type

```

(engine) car = new (car());
sout(car.a); // 10
sout(car.b); error
car.start; // car.start

```

reference type is engine, tho jo method, variable engine me hoga whi access hoga.

```

(car) car = new engine ();
sout(car.a);
sout(car.b);
car.start;

```

```

engine car = new engine();
sout(car.a);
sout(car.b);
car.start;

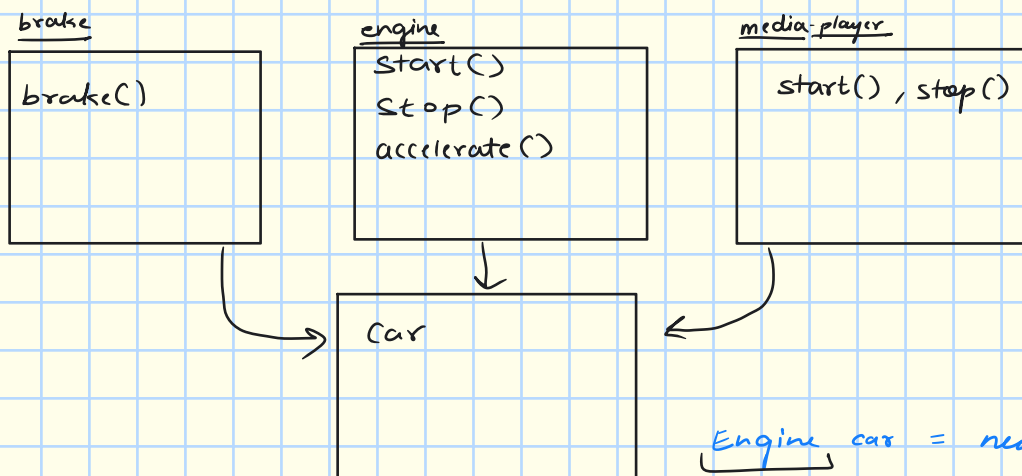
```

# # Interfaces ★ Interface notes

- by default variables in interfaces are **static & final**
- by default methods are abstract.

→ multiple inheritance → have issue if engine & media-player both have same methods then child will get difficulty to which ones method I need to access, **So, Interfaces** resolve this by focusing on what a class should do rather than how it does it.

ex:-



Now I want one car which have all these property of stop, engine, media-player, but Java not support multiple inheritance, **So** here come **interf** in game.

**Engine** car = new **Car()**;  
what things you access depend on this  
which one of those versions to access depends on this.

Code

```
public interface Brake {
    void brake();
}
```

```
public interface Engine {
    void start();
    void stop();
}
```

```
public interface Media {
    void start();
    void stop();
}
```

```
public class Car implements Engine, Brake, Media {
    @Override
    public void start() {
        System.out.println("I start engine like normal car");
    }

    @Override
    public void stop() {
        System.out.println("I stop engine like normal car");
    }

    @Override
    public void accelerate() {
        System.out.println("I accelerate like normal car");
    }

    @Override
    public void brake() {
        System.out.println("I brake like normal car");
    }
}
```

→ **public class main {**  
**psvm(...)** {

**Engine** car = new **Car()**;  
car.start(); // I start engine like normal car

**Media** car2 = new **Car()**;  
car2.start(); // I start engine like normal car.

How weird to start media its result the same thing when start engine.

## # Seperate classes in Same Interfaces.

### Brake.java

```
public interface Brake {  
    void brake();  
}
```

→ Interfaces

### Engine.java

```
public interface Engine {  
    void start();  
    void stop();  
}
```

### PowerEngine.java

```
public class PowerEngine implements Engine() {  
    @Override  
    public void start() {  
        System.out.println("Start Power Engine");  
    }  
    @Override  
    public void stop() {  
        System.out.println("Stop Power Engine");  
    }  
}
```

### ElectricEngine.java

```
public class ElectricEngine implements Engine() {  
    @Override  
    public void start() {  
        System.out.println("Start Electric Engine");  
    }  
    @Override  
    public void stop() {  
        System.out.println("Stop Electric Engine");  
    }  
}
```

### CDPlayer.java

```
public class CDPlayer implements Media {  
    @Override  
    public void start() {  
        System.out.println("Music Start");  
    }  
    @Override  
    public void stop() {  
        System.out.println("Music stop");  
    }  
}
```

### NiceCar.java

→ gonna use those classes which implemented interfaces.

```
public class NiceCar {  
    private Engine engine;  
    private Media media = new carCDPlayer();  
  
    public NiceCar() {  
        engine = new PowerEngine();  
    }  
  
    public void startEngine() {  
        engine.start();  
    }  
  
    public void stopEngine() {  
        engine.stop();  
    }  
  
    public void startMusic() {  
        media.start();  
    }  
  
    public void stopMusic() {  
        media.stop();  
    }  
  
    public void updateEngine() {  
        engine = new ElectricEngine();  
    }  
}
```

### main.java

```
public class main {  
    public static void main(String[] args) {  
        NiceCar car = new NiceCar();  
        car.startEngine();  
        car.startMusic();  
        car.upgradeEngine();  
        car.startEngine();  
    }  
}
```

Issue resolved now. startEngine → start Engine  
startMusic → start Music

## # Extending interfaces.

```
public interface A {  
    void fun();  
}
```

```
public interface B extends A {  
    void greet();  
}
```

```
public class Main implement B {  
    @Override  
    void greet() { System.out.println("Hello"); }  
    @Override  
    void fun() { System.out.println("Have fun"); }  
}
```

⇒ Here B inherit A, So when B get implement then we have to override method of B & also those methods which B inherits.

## # Annotations

- @Override → It's also a interface internally  
↳ Annotation

## # Static interface method

ex:-

```
public interface A {  
    // static interface methods should always have  
    // a body.  
    static void greeting() {  
        sout("Hello");  
    }  
}
```

public class main {  
 psvm(...) {  
 A.greeting();  
 }  
}

### Imp. Note

Inheritance means like if you inherit it, you have to override. Overriding means it depends on objects because it's runtime polymorphism, and static methods do not depend on objects. That's why they cannot be inherited.

→ Why static methods not override?

## # Nested Interfaces

```
public class A {  
    // nested interface  
    public interface NestedInterface {  
        boolean isOdd(int num);  
    }  
}
```

```
class B implements A.NestedInterface {  
    @Override  
    public boolean isOdd(int num) {  
        return (num % 2) == 1;  
    }  
}
```

```
public class main {  
    psvm(...) {  
        B obj = new B();  
        sout(obj.isOdd(6));  
    }  
}
```

